

CS3002E Compiler Design

Monsoon 2025

Lecture #24

Intermediate Code Generation - Arithmetic Expressions, Assignment

Department of Computer Science & Engineering
NIT Calicut

Three-Address Code: Recap

- Instructions of the form $x=y \text{ op } z$ with three addresses - two for the operands and one for the result.
- At most one operator on the right side of an instruction.
- Temporary names (generated by the compiler) used for intermediate values.

Three-Address Code: Addresses

An *address* in a 3-address instruction can be

- a name in the source program
- a constant
- a compiler-generated temporary

e.g. the instruction $t_1 = a + b$ contains 3 addresses: a and b are names in the source program, t_1 is a temporary.

Note: In the implementation, a name is replaced by a pointer to its *Symbol Table* entry.

Three-Address Code: example

The assignment $x = a + b * c$ is translated to the following sequence of 3-address instructions:

$$t_1 = b * c$$

$$t_2 = a + t_1$$

$$x = t_2$$

t_1 , t_2 are compiler generated temporaries

Generation of 3-address code - expressions

Production	Semantic Rules
$E \rightarrow E_1 + E_2$	$E.addr = newTemp()$ $E.code = E_1.code \parallel E_2.code \parallel$ $gen(E.addr \text{ ' = ' } E_1.addr \text{ ' + ' } E_2.addr)$
$E \rightarrow id$	$E.addr = id.entry$ $E.code = \text{' '}$

E.code: 3-address code for E

E.addr: the address¹ that will hold the value of E

¹the address can be a name, a constant or a compiler-generated temporary

Generation of 3-address code - expressions

Production	Semantic Rules
$E \rightarrow E_1 + E_2$	$E.addr = newTemp()$ $E.code = E_1.code \parallel E_2.code \parallel$ $gen(E.addr \text{ ' = ' } E_1.addr \text{ ' + ' } E_2.addr)$
$E \rightarrow id$	$E.addr = id.entry$ $E.code = \text{' '}$

- **id.entry**: pointer to the symbol table entry for **id** (denoted by the name of the **id** for convenience)
- *newTemp()*: returns a distinct temporary name
- *gen()*: generates and returns a 3-address instruction, concatenating its arguments (quoted arguments as such, expressions replaced by value)

SDD for translating Assignment statement

Production	Semantic Rules
$S \rightarrow id = E$	$S.code = E.code \parallel gen(id.entry \text{ ' = ' } E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = newTemp()$ $E.code = E_1.code \parallel E_2.code \parallel$ $gen(E.addr \text{ ' = ' } E_1.addr \text{ ' + ' } E_2.addr)$
$E \rightarrow -E_1$	$E.addr = newTemp()$ $E.code = E_1.code \parallel$ $gen(E.addr \text{ ' = ' 'minus' } E_1.addr)$
$E \rightarrow (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$E \rightarrow id$	$E.addr = id.entry$ $E.code = \text{ ' ' }$

Exercise

1. Write the 3-address code generated for the following instructions, as per the given SDD:

$$x = y$$

$$x = a + b$$

$$x = a + -(c + d)$$

2. Extend the SDD by adding production $E \rightarrow E * E$ with the required semantic rule. Write the 3-address code generated for $x = (a + b) * (c + d)$

References

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU Chapter 6 - Introduction, 6.4.1.